

CHATBOT SPECIALIZE IN AI

Chatbot based on LLM + RAG + Langchain to build chatbot that work well with AI related queries. I use datasets from huggingface llama2 arxiv and AI modern approach book as external data.

Tech stacks:

Python

Pandas

Numpy

NLTK

spaCy

regex

logging

Pinecone

Langchain

- Langchain_community
- HuggingFaceEndpoing
- HuggingFaceEmbedding
- PineconeVectorStore
- PyPDFDirectoryLoader
- RecursiveCharacterTextSplitter

Pypdf

PyMuPDF

Datasets

Streamlit

Features

- User can send their query into chatbot and they get accurate answer based on the data in the vectordb, this helps to reduce huallacination in LLM.
- They can select in which format they want response like beginner, intermediate, expert
- Delete all previous chat and start new
- Select their preferred model to response

Working

Workflow:

Query -> vectordb -> augmented prompt -> send to selected model -> response

Project Planning

Project Overview

- **Goal:** Build a functional prototype of an AI chatbot specialized in AI-related queries using a large language model (LLM), retrieval-augmented generation (RAG), and Langchain.
- **Key Features (Simplified):**
 - Basic query-response functionality with a single response format.
 - One data source and one LLM for faster implementation.
 - Minimal user interface for testing and demonstration.
- **Tech Stack:** Python, Pandas, Numpy, Pinecone, Langchain, Hugging Face, Streamlit.

Tasks:

➤ **Data Loading**

- Use a single data source: either the Hugging Face Llama2 Arxiv dataset or the "AI Modern Approach" book PDF. (Recommendation: Start with the book for easier text extraction.)
- Simplify preprocessing:

- Extract text using PyPDF (accept some noise, like formatting artifacts).
 - Remove only obvious irrelevant sections (e.g., references or footnotes).
- Store the text in a simple format (e.g., a single text file or list of paragraphs).

➤ **Vector Database setup**

- Use a lightweight embedding model (e.g., all-MiniLM-L6-v2 from sentence-transformers) for speed.
- Chunk the text into larger sections (e.g., pages or chapters) to reduce the number of embeddings.
- Embed a subset of the data (e.g., first few chapters or a small sample of papers).
- Set up Pinecone and upsert the embeddings into the vector database.

• **Chatbot Logic and RAG Integration**

- Define a simple workflow: Query → Vector Database → Augmented Prompt → LLM → Response.
- Use a basic prompt template (e.g., "Based on the following context: {retrieved_documents}, answer the query: {user_query}").
- Implement RAG to retrieve the top 1-2 most relevant documents from the vector database.
- Integrate a single, smaller LLM (e.g., distilgpt2 or gpt2-medium from Hugging Face) for faster processing.

➤ **Minimal User Interface**

- Use Streamlit to build a minimal interface with:

- A text input field for user queries.
- A display area for chatbot responses.
- No advanced options like response format selection or model choice.
- Connect the UI to the chatbot logic developed in Phase 3.

➤ **Lightweight Testing and Iteration**

- Manually test the chatbot with 5-10 sample AI-related queries.
- Check for basic functionality (e.g., retrieves relevant documents, generates coherent responses).
- Fix any major issues (e.g., errors in retrieval or prompt formatting).
- If time permits, note a simple metric like response time or basic usability feedback